24

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 2

가드와 실행 컨텍스트

제2부 NestJS 핵심 · 6장 요청 파이프라인

들여보낼지 정하는 관문과, 그 관문이 보는 요청의 정보



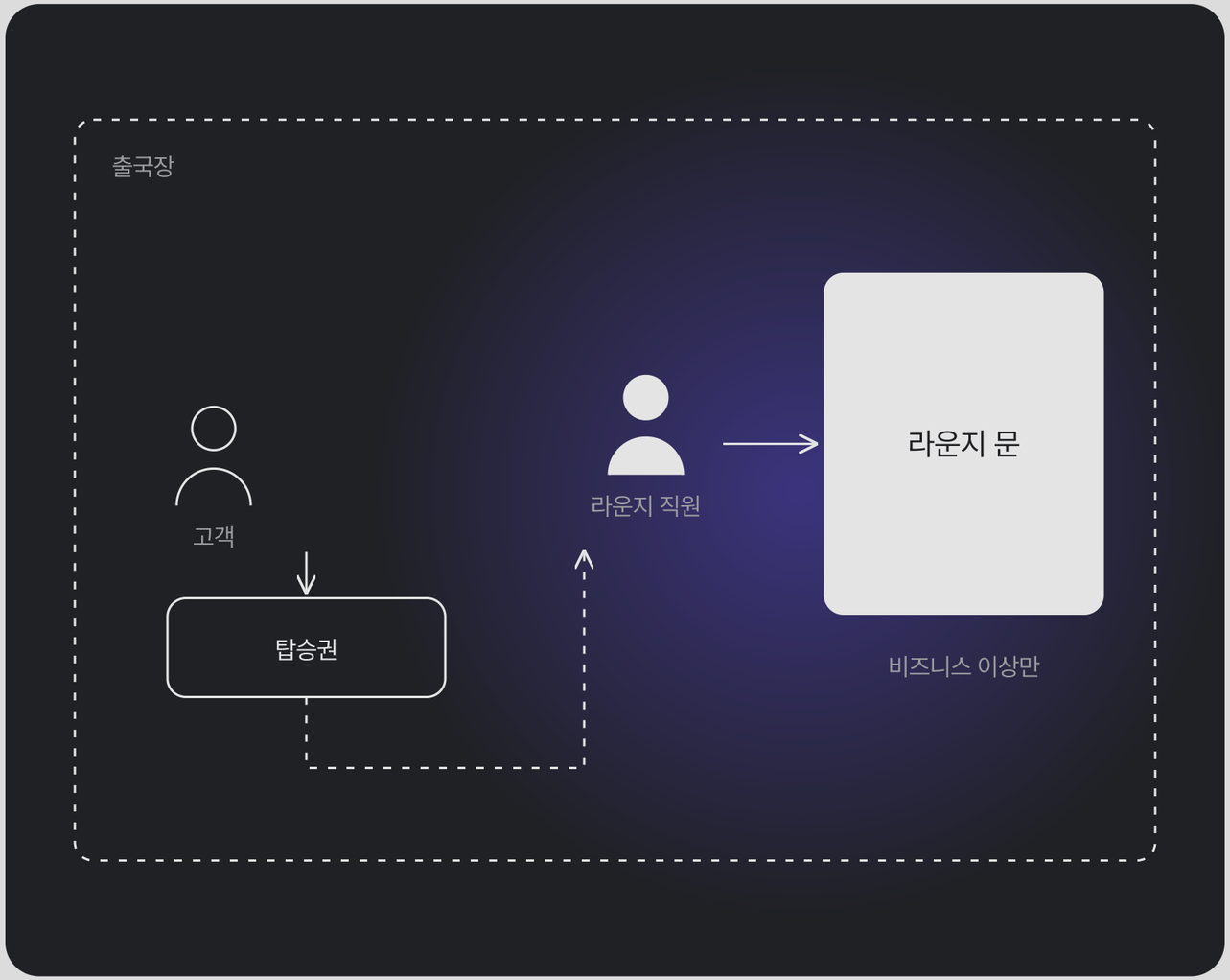
© 한상훈 NestJS 백엔드 마스터

라운지 입구 직원

→ 확인하는 것: 탑승권 등급(비즈니스 이상)

→ 국내선 표인지 국제선 표인지: 표를 받아 들고서야 앎

→ 출국장 입구가 아니라 라운지 문 앞에 서 있음



© 한상훈 NestJS 백엔드 마스터

미들웨어로 막은 매표소

```ts
// 매표소 경로면 직원 키를 확인한다 (경로 문자열 비교)
export function boxOfficeKey(req: Request, res: Response, next: NextFunction) {
  if (req.path.startsWith('/box-office') && req.header('x-staff-key') !== process.env.STAFF_KEY) {
    res.status(401).json({ message: '직원 키가 필요합니다' });
    return;
  }
  next();
}
```

→ main.ts에서 `app.use(boxOfficeKey)`

© 한상훈 NestJS 백엔드 마스터

대문자로 보낸 판매 요청

curl

```
$ curl -X POST localhost:4024/box-office/performances/1/seats/A-1-1/sell
{"message":"직원 키가 필요합니다"} [401]

$ curl -X POST localhost:4024/Box-Office/performances/1/seats/A-1-1/sell
{"id":"A-1-1","label":"A구역 1열 1번","status":"SOLD"} [201]
```

→ 직원 키 없이 좌석 판매 성공

→ 로그에는 정상 판매 201로만 남음

© 한상훈 NestJS 백엔드 마스터

검사와 라우팅이 문자열을 다르게 읽음

경로	미들웨어 검사	Express 라우팅
`/box-office/…`	매표소로 판단, 키 확인	판매 핸들러
`/Box-Office/…`	매표소 아님, 통과	판매 핸들러 (대소문자 무시)
`/box-office/…/sell/`	매표소로 판단	판매 핸들러 (끝 슬래시 무시)

→ 미들웨어에서 라우터 규칙을 흉내: 대소문자, 끝 슬래시, 인코딩… 끝이 없음

데이터 블록: Express 5.3.0 기본 라우팅: 대소문자 구분 안 함, 끝 슬래시 무시 (2026-10-26 실측), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

가드

- → 이 요청을 핸들러까지 들여보낼지 정하는 클래스
- → `CanActivate` 구현, `canActivate(context)` 하나
- → 참이면 통과, 거짓이면 403
- → 반환값: `boolean`, `Promise<boolean>`, `Observable<boolean>`
- → 라우트가 정해진 뒤, 그 라우트에 건 가드만 실행

© 한상훈 NestJS 백엔드 마스터

StaffKeyGuard

staff-key.guard.ts

```ts
@Injectable()
export class StaffKeyGuard implements CanActivate {
  private readonly logger = new Logger('StaffKeyGuard');

  constructor(private readonly staffService: StaffService) {}

  async canActivate(context: ExecutionContext): Promise<boolean> {
    const req = context.switchToHttp().getRequest<Request & { staff?: Staff }>();
    this.logger.log(`${context.getType()} ${context.getClass().name}.${context.getHandler().name}`);

    const staff = await this.staffService.verify(req.header('x-staff-key'));
    if (!staff) throw new UnauthorizedException('직원 키가 없거나 맞지 않습니다');
    if (staff.role === 'planner') throw new ForbiddenException(`${staff.name}님은 매표 권한이 없습니다`);

    req.staff = staff;
    return true;
  }
}
```

© 한상훈 NestJS 백엔드 마스터

@UseGuards

box-office.controller.ts

```ts
@Controller('box-office/performances/:performanceId/seats')
@UseGuards(StaffKeyGuard)
export class BoxOfficeController {
  constructor(private readonly seatsService: SeatsService) {}
  // sell …
}
```

→ 클래스 위: 이 컨트롤러의 모든 핸들러

→ 메서드 위: 그 핸들러만

→ 가드 여러 개: 적은 순서대로, 하나라도 막으면 끝

→ SeatsModule imports에 StaffModule 추가 (가드가 `StaffService`를 주입받음)

© 한상훈 NestJS 백엔드 마스터

같은 대문자 요청

curl

```
$ curl -X POST localhost:4024/box-office/performances/1/seats/A-1-1/sell
{"type":"about:blank","title":"Unauthorized","status":401,
 "detail":"직원 키가 없거나 맞지 않습니다", …} [401]

$ curl -X POST localhost:4024/Box-Office/performances/1/seats/A-1-1/sell
{"type":"about:blank","title":"Unauthorized","status":401,
 "detail":"직원 키가 없거나 맞지 않습니다", …} [401]
```

→ 경로 문자열 비교 없음: 라우터가 고른 핸들러에 붙은 가드가 실행

© 한상훈 NestJS 백엔드 마스터

실행 컨텍스트

→ `ArgumentsHost`	이 요청의 인자 묶음 (HTTP면 req, res, next)
→ `ExecutionContext`	= ArgumentsHost + 두 메서드
→ `getClass()`	이 요청을 처리할 컨트롤러 클래스
→ `getHandler()`	이 요청을 처리할 핸들러 메서드
→ 받는 관문	가드·인터셉터가 받는 인자

© 한상훈 NestJS 백엔드 마스터

가드가 찍은 로그

서버 로그

```
LOG [StaffKeyGuard] http BoxOfficeController.sell
LOG [HTTP] 492b1def-… POST /box-office/performances/1/seats/A-1-1/sell 401 10.3ms
LOG [StaffKeyGuard] http BoxOfficeController.sell
LOG [HTTP] ceeee3a6-… POST /Box-Office/performances/1/seats/A-1-1/sell 401 13.2ms
```

→ 두 요청 모두: 같은 클래스, 같은 메서드

© 한상훈 NestJS 백엔드 마스터

getType과 switchTo

getType()	요청 종류	꺼내는 방법
`'http'`	HTTP 요청	`switchToHttp().getRequest()`
`'rpc'`	마이크로서비스 메시지	`switchToRpc().getData()`
`'ws'`	웹소켓 메시지	`switchToWs().getClient()`

→ 같은 가드를 여러 종류의 요청에 쓸 때 `getType`으로 갈래

→ `getHandler()`: 메서드 이름이 아니라 함수 자체, 로그의 sell은 함수의 name

© 한상훈 NestJS 백엔드 마스터

false를 돌려주면

서버 로그 · 응답

```
1 미들웨어 GET /trace/7
2 가드 (params.id: string "7")
7 예외 필터 (ForbiddenException)
{"type":"about:blank","title":"Forbidden","status":403,
 "detail":"Forbidden resource","instance":"/trace/7"}
```

→ 거짓을 돌려준 가드: Nest가 `ForbiddenException('Forbidden resource')`

→ 이유와 상태 코드를 고를 수 없음

© 한상훈 NestJS 백엔드 마스터

401과 403

상태 코드	뜻	클라이언트가 할 일
401 Unauthorized	누구인지 모름 (키 없음, 틀린 키)	키 입력·로그인 화면으로
403 Forbidden	누구인지 알지만 이 일은 안 됨	권한 없음 안내, 다시 시도 안 함

→ 전부 403이면: 키를 잘못 넣은 직원도 "권한 없음"만 봄

→ 전부 401이면: 권한 없는 직원이 키 입력 화면을 계속 반복

"상태 코드는 응답을 받은 프로그램이 다음에 할 일을 정하는 숫자입니다."

© 한상훈 NestJS 백엔드 마스터

직원 키 세 개

curl

```
$ curl -X POST …/sell -H 'x-staff-key: key-nope'
{"title":"Unauthorized","status":401,"detail":"직원 키가 없거나 맞지 않습니다", …}

$ curl -X POST …/sell -H 'x-staff-key: key-park'
{"title":"Forbidden","status":403,"detail":"박기획님은 매표 권한이 없습니다", …}

$ curl -X POST …/sell -H 'x-staff-key: key-kim'
{"id":"A-1-1","label":"A구역 1열 1번","status":"SOLD","soldBy":"김매표"} [201]
```

키	직원	역할
key-kim	김매표	판매원
key-lee	이실장	실장
key-park	박기획	기획팀

© 한상훈 NestJS 백엔드 마스터

StaffService.verify

staff.service.ts

```ts
@Injectable()
export class StaffService {
  private readonly staff = new Map<string, Staff>([
    ['key-kim', { name: '김매표', role: 'clerk' }],
    ['key-lee', { name: '이실장', role: 'manager' }],
    ['key-park', { name: '박기획', role: 'planner' }],
  ]);

  // DB 조회를 흉내 낸 비동기 메서드
  async verify(key: string | undefined): Promise<Staff | undefined> {
    await new Promise((r) => setTimeout(r, 5));
    return key ? this.staff.get(key) : undefined;
  }
}
```

→ StaffModule: providers와 exports에 `StaffService`

© 한상훈 NestJS 백엔드 마스터

await를 빠뜨린 가드

```
canActivate(context: ExecutionContext): boolean {
  const req = context.switchToHttp().getRequest<Request>();
  return !!this.staffService.verify(req.header('x-staff-key')); // await 빠뜨림
}
```

→ verify의 반환값: Promise 객체

→ !!Promise → 늘 true

© 한상훈 NestJS 백엔드 마스터

아무 키로나 판매

curl

```
$ curl -X POST …/seats/A-1-3/sell -H 'x-staff-key: key-nope'
{"id":"A-1-3","label":"A구역 1열 3번","status":"SOLD"} [201]

$ curl -X POST …/seats/A-1-4/sell
{"id":"A-1-4","label":"A구역 1열 4번","status":"SOLD"} [201]
```

→ 에러 없음, 경고 없음, 판매 성공

→ 고친 코드: `async canActivate`, `await verify`

→ 가드 시험: 통과보다 막히는 경우(틀린 키, 키 없음)를 먼저

© 한상훈 NestJS 백엔드 마스터

req.staff

box-office.controller.ts

```
@Post(':seatId/sell')
sell(
  @Param('performanceId', ParseIntPipe) performanceId: number,
  @Param('seatId') seatId: string,
  @Req() req: Request & { staff: Staff },
) {
  return { ...this.seatsService.sell(performanceId, seatId), soldBy: req.staff.name };
}
```

→ 가드가 찾은 직원을 핸들러가 다시 조회하지 않음

© 한상훈 NestJS 백엔드 마스터

전역으로 거는 두 방법

방법	코드	StaffService 주입
main.ts	`app.useGlobalGuards(new StaffKeyGuard(new StaffService()))`	없음, StaffService를 직접 new (Nest가 만든 것과 별개)
모듈 프로바이더	`{ provide: APP_GUARD, useClass: StaffKeyGuard }`	있음

© 한상훈 NestJS 백엔드 마스터

APP_GUARD로 걸면

app.module.ts

```
@Module({
  imports: [ /* … */ StaffModule],
  providers: [AppService, { provide: APP_GUARD, useClass: StaffKeyGuard }],
})
export class AppModule {}
```

○ ○ ○ curl · 서버 로그

```
$ curl localhost:4024/concerts
{"title":"Unauthorized","status":401,"detail":"직원 키가 없거나 맞지 않습니다", …}
$ curl localhost:4024/performances/1/seats
{"title":"Unauthorized","status":401, …}

LOG [StaffKeyGuard] http ConcertsController.findAll
LOG [StaffKeyGuard] http SeatsController.findAll
```

© 한상훈 NestJS 백엔드 마스터

지금은 컨트롤러에

→ 전역 가드: 모든 핸들러에 실행, 고객 API까지 401

→ `APP_GUARD`는 어느 모듈에 적어도 서버 전체

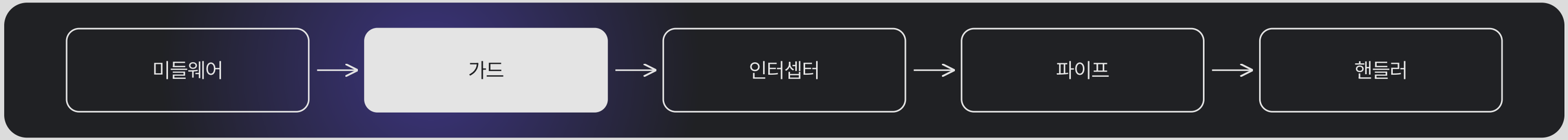
→ 전역에 걸려면: 핸들러마다 "직원 전용" 여부를 알아야 함

→ 지금 코드: BoxOfficeController에 `@UseGuards(StaffKeyGuard)`

→ 단점: 새 매표소 컨트롤러를 만들 때 `@UseGuards`를 빠뜨릴 수 있음

© 한상훈 NestJS 백엔드 마스터

가드 시점의 요청



→ 가드가 보는 경로 값: 문자열 (예: `params.id = "7"`)

→ 가드가 보는 본문: 검증·변환 전

→ 가드에서 본문의 금액, 좌석 번호를 믿고 판단하지 않음

→ 가정: "10만 원 이상은 실장만"을 가드에서 `amount`로 판단 → 문자열 "9"가 통과

→ 본문에 달린 규칙: 검증이 끝난 값을 받는 서비스에서

© 한상훈 NestJS 백엔드 마스터

가드가 할 일

가드에 둠	가드에 두지 않음
이 사람이 누구인가 (401)	본문 값 검사 (파이프)
이 사람이 이 핸들러를 써도 되는가 (403)	좌석이 남았는가 (서비스)
찾은 사람을 req에 싣기	응답 바꾸기 (인터셉터)

→ 가드는 요청마다 실행: 판매 요청 1ms 미만 → 가드 뒤 10~18ms (직원 조회 5ms 대기 포함)

→ 고객 API 전체에 매번 DB 조회 가드 → 오픈 순간 조회 수십만 번

데이터 블록: 강사 PC 실측, 가드 포함 판매 요청 10.3~18.6ms (2026-10-26), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

가드

"가드는 이 요청이 어느 핸들러로 가는지 알고 나서, 들여보낼지 말지만 정합니다."